Performance Analysis Document

Summary

Overview

Comparison of the performance of two different algorithms for finding the maximum element of an array with GPU acceleration. Int, long long int, float and double data types' performance are compared separately.

The two algorithms compared are:

- Atomic max: use of atomicMax CUDA function (for int and long long int types) or atomicCAS CUDA function (for float and double types) for updating a device memory address where the maximum value will be stored atomically. This means that there exist no race conditions between different threads writes.
- Reduction: Calculation of local maximum elements. It starts dividing the array into pairs, so the maximum value searching between these pairs can be done in parallel. Then it divides the "local maximums" array by 2, so the maximum value searching of the already calculated local maximum values can be done in parallel for every pair. It continue repeating this step until only 1 element is left, which will be the maximum value of the array.

Benchmark environment

- GPU model: RTX 3070 Laptop
- CPU model: Intel core i7 12700H
- GPU compute capability: 8.6
- GPU driver version: 595.84
- CUDA version: 13.3
- g++ version: 15.2.0
- Cmake version: 4.2.3
- OS: Ubuntu v26.04.1

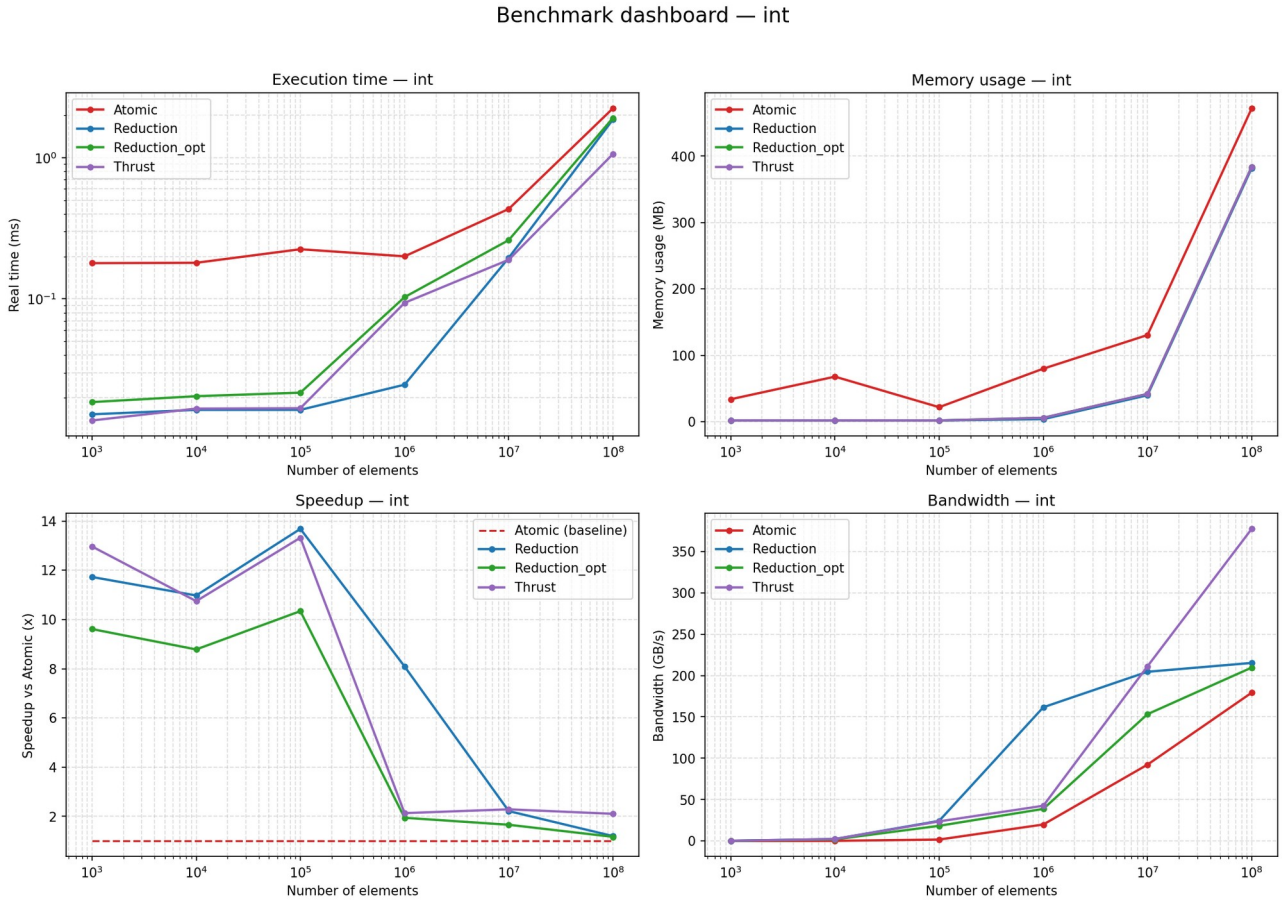
Methodology

- Array sizes tested: 10^3 – 10^8
- Array data types tested: int, long long int, float and double
- Number of executions per benchmark: A minimum of 0.5s of execution repeated 10 times
- Measured statistics
  - Primary: real execution time, device memory used

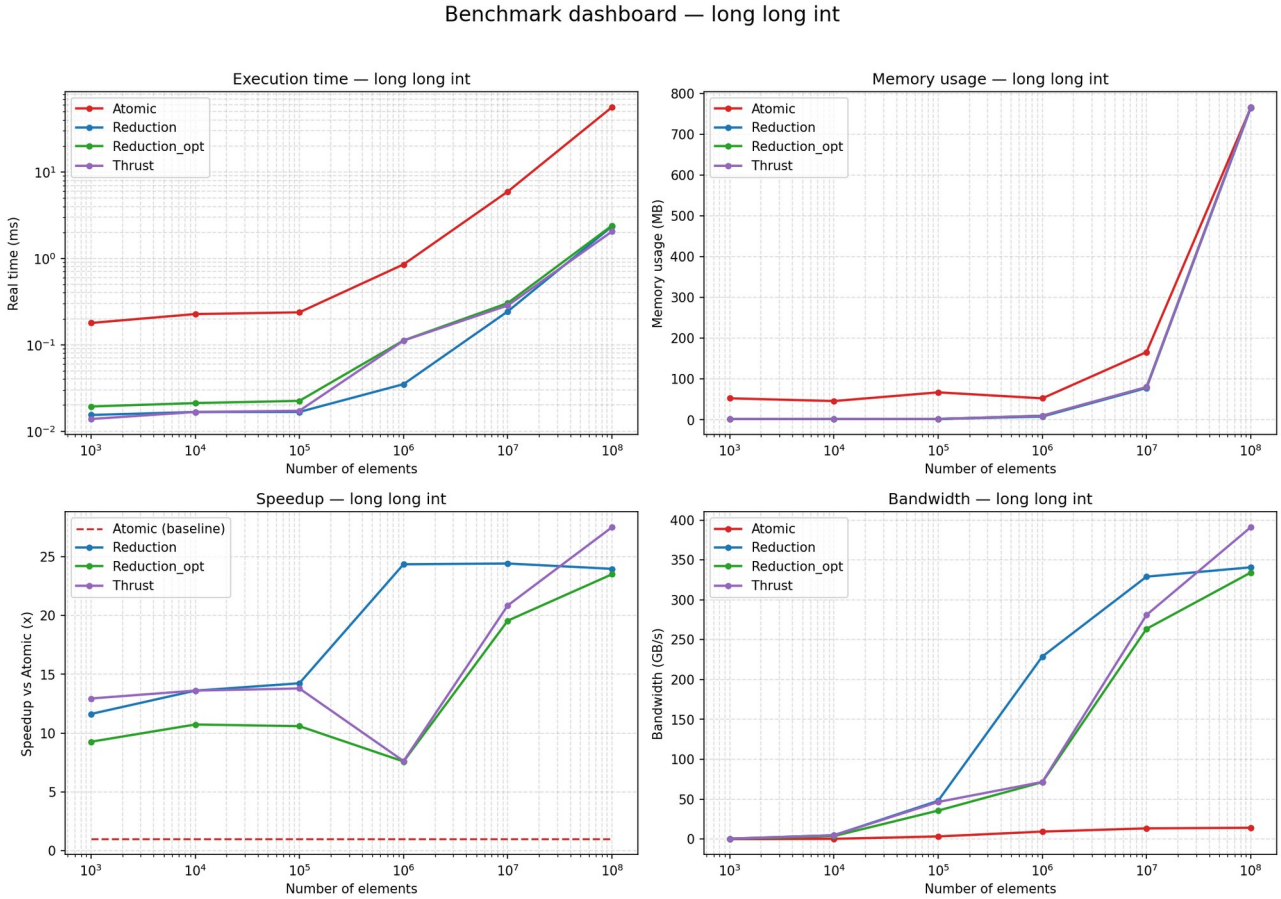
    - Derivated: throughput (bytes/s), speedup vs atomic method
- GPU frequency fixed

Results

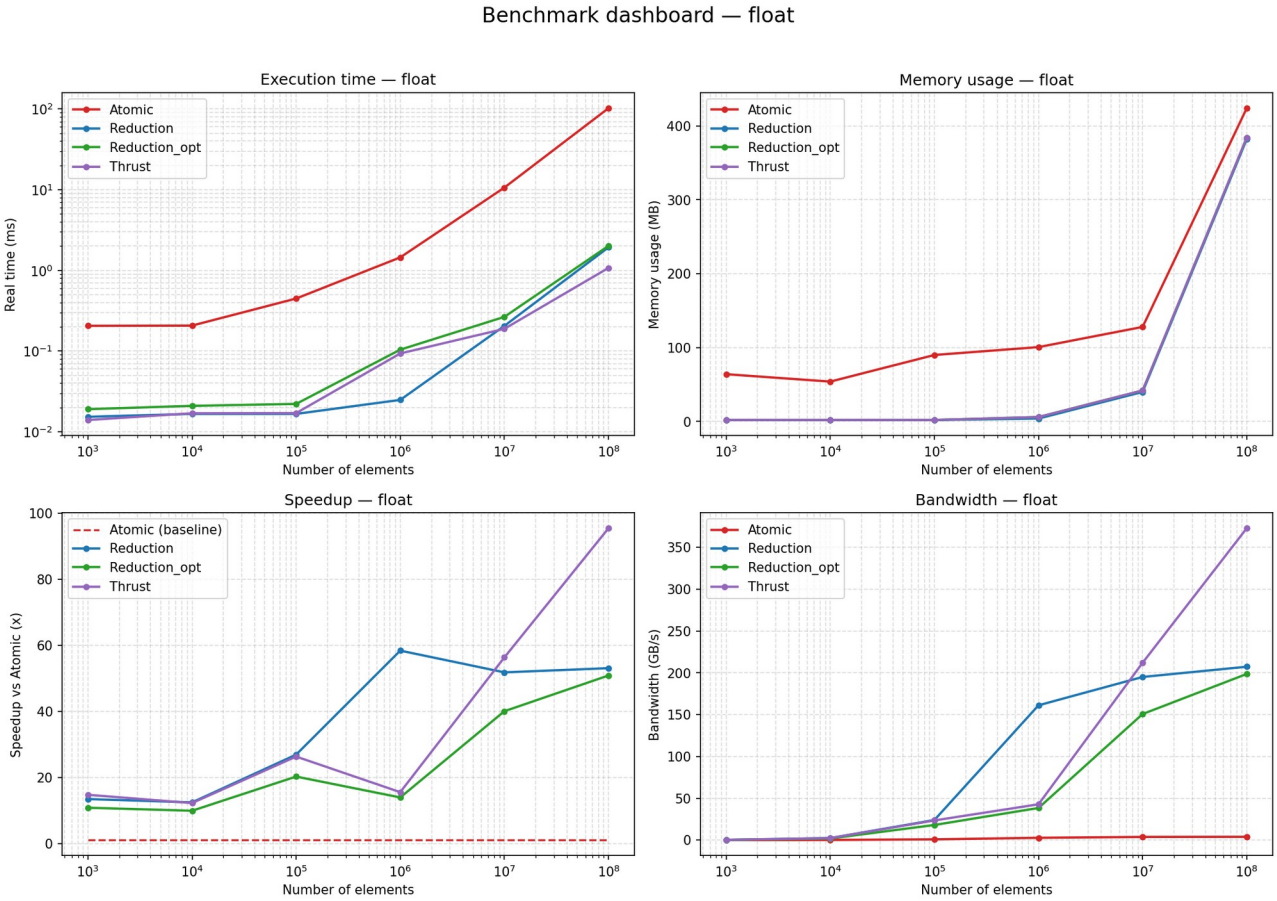
int


Benchmark dashboard — int
Execution time — int
Real time (ms)
Number of elements
Memory usage — int
Memory usage (MB)
Number of elements
Speedup — int
Speedup vs Atomic (x)
Number of elements
Bandwidth — int
Bandwidth (GB/s)
Number of elements
Atomic
Reduction
Reduction_opt
Thrust
Atomic (baseline)

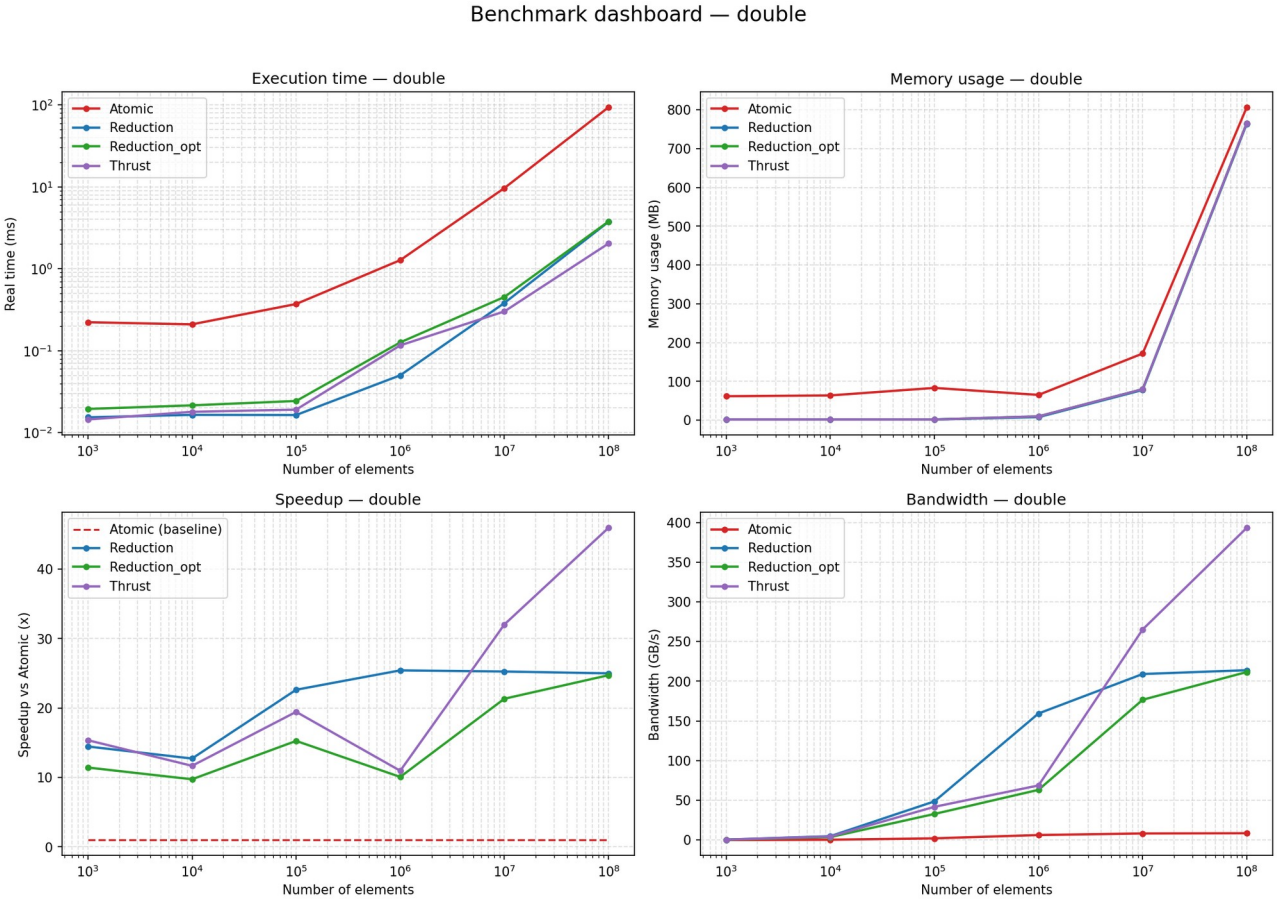
long long int



float


Benchmark dashboard — float
Execution time — float
Real time (ms)
Number of elements
Memory usage — float
Memory usage (MB)
Number of elements
Speedup — float
Speedup vs Atomic (x)
Number of elements
Bandwidth — float
Bandwidth (GB/s)
Number of elements
Atomic
Reduction
Reduction_opt
Thrust
Atomic (baseline)

double


Benchmark dashboard — double
Execution time — double
Real time (ms)
Number of elements
Memory usage — double
Memory usage (MB)
Number of elements
Speedup — double
Speedup vs Atomic (x)
Number of elements
Bandwidth — double
Bandwidth (GB/s)
Number of elements
Atomic
Reduction
Reduction_opt
Thrust
Atomic (baseline)

Analysis

Atomic

The execution time of the atomic method is higher than the other ones, reaching up to x60 speedup.

This phenomenon is due to the synchronization cost of updating the device memory address where the maximum value will be stored.

When using floating point types (float and double), this cost is even higher, because this synchronization is not done by the hardware, but by the software, needing to recalculate by each thread the maximum between its value and the one stored by another thread atomically whenever it has been overtaken in the storage.

Reduction vs Reduction_opt

The reduction_opt method is slower than the reduction.

This behaviour can be explained by the execution timeline analysed with nvidia nsight systems.
Reduction:

#	Name	Start	Duration	TID
1	cuModuleGetLoadingMode	0,3059s	1,182 µs	28234
2	cudaMalloc	0,306013s	104,181 ms	28234
3	cudaMemcpyAsync	0,410206s	1,416 ms	28234
4	cudaStreamSynchronize	0,411623s	1,074 ms	28234
5	cuLibraryLoadData	0,412712s	168,685 µs	28234
6	cuLibraryGetKernel	0,412881s	263 ns	28234
7	cuKernelGetName	0,412883s	225 ns	28234
8	reduction_max_gpu	0,412883s	114,202 µs	28234
10	cuKernelGetName	0,412998s	106 ns	28234
11	reduction_max_gpu	0,412999s	3,533 µs	28234

Reduction_opt:

#	Name	Start	Duration	TID
1	cuModuleGetLoadingMode	0,306516s	1,198 µs	29166
2	cudaMalloc	0,306632s	103,487 ms	29166
3	cudaMemcpyAsync	0,41013s	1,433 ms	29166
4	cudaStreamSynchronize	0,411564s	1,067 ms	29166
5	cudaMalloc	0,412642s	51,305 µs	29166
6	Runtime Triggered Module Loading	0,412702s	70,632 µs	29166
7	Lazy Function Loading	0,413s	10,008 µs	29166
8	cuLibraryGetKernel	0,413017s	759 ns	29166
9	cuKernelGetName	0,413019s	170 ns	29166
10	static_kernel	0,413019s	22,914 µs	29166

For example, in the case of float data type and 10^6 elements, it is being added an extra cudaMalloc for auxiliar memory (and its corresponding cudaFree), which explains the fixed displacement of the green line (reduction_opt) vs the blue line (reduction).

It also explains why it disappears with 10^8 elements. A large number of elements causes a long execution time of the kernel, which hides the extra malloc time because it is a small percentage of the time.

The opt method enables coalescent reads, which should be accelerating the kernel, and it does, but the optimization is only applied on the second and consecutives runs of the kernel.

When using for example, 100 million elements, on the first run of the kernel there will be executed 100 million reads. On the second run 97657 and on the third run only 96 reads.
The most part of the execution time is taken by the first run, because it has to process a lot of more elements than the second run (~x1024) and the third run (~x1041666), so the optimization is not being perceived.

64 bit data types

In the case of 64 bit types (long long int and double), the bottleneck is the number of cores that they can use, which is much lower than 32 bit types. In the case of long long int type, the comparison is executed faster than the comparison between double type elements, so it explains why it processes 350 GB/s instead of the 200GB/s that are being processed when using the other data types.

How to reproduce

For reproducing the benchmarks it is needed to have the same benchmark environment, and follow the following steps:

- Go to directory:

  max_search_comparison/

- Cmake configuration:

  cmake -B build -DCMAKE_BUILD_TYPE=Release

- Compilation:
  cmake --build build

- Fixing the frequency of the GPU:

  sudo nvidia-smi -lgc 2100,2100

- Executing benchmarks:

  ./scripts/run_benchmarks.sh

- Restoring the GPU frequency:

  sudo nvidia-smi -rgc

All the plots will be stored in max_search_comparison/plots directory.

Limitations

- Explanation of the gap between blue and green lines in execution time when 10^6 size array is used.
- No comparison against CPU baseline
- Only 8.6 compute capability tested

Conclusions

In conclusion, the correct selection of an algorithm for calculating the maximum number of an array is reduction, which is also the one used by thrust implementation.
As mentioned, the speedup of this algorithm in comparison with the atomic method is very high, and scales with the number of elements of the array.